Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Технологии и инструментальные средства проектирования интеллектуальных систем»
на тему:

Разработка модуля адаптации знаний для системы NIKA

Студенты гр. 321701 И. А. Политыко
В. А. Лукашов
Руководитель Н. В. Гракова

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений 3
1 Постановка задачи 4
2 Документация агента 5
2.1 Условие инициирования и результат 5
2.2 Входы и выходы 5
2.3 Функционал агента 6
2.4 Схема алгоритма агента 6
3 Интеграция и работа модуля в системе 8
4 Тестирование 10
Заключение 12
Список использованных источников 13

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

NIKA	— Intelligent Knowledge-driven Assistant, интеллектуальный ассистент на базе OSTIS;
БЗ	— База знаний;
SCs	— Symbolic Code compact, текстовый язык представления SC-кода;
агент	— Программный модуль, инициируемый событием в SC-памяти;
действие	— Узел класса `actions_class`, определяющий работу агента;
намерение	— Класс сообщения, определяемый классификатором (intent);
сущность	— Элемент БЗ, выделенный из сообщения пользователем (entity);
заметка	— Отношение «заметка» (`nrel_note`), текстовое описание сущности;
основной идентификатор	— Отношение `nrel_main_idtf`, имя сущности на языке;
MediaWiki API	— HTTP-интерфейс Википедии, отвечает в формате JSON;
JSON	— JavaScript Object Notation, формат передачи данных;
witai-shim	— Модуль классификации сообщений системы NIKA.

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение принципов работы и проектирования решателей задач интеллектуальных систем, освоение навыков проектирования и реализации программ с помощью различных видов представления знаний и моделей решения задач, научиться анализировать и выводить знания.

Задача работы состоит в разработке модуля адаптации знаний для системы NIKA: агента, который получает знания из внешнего источника, погружает их в базу знаний и позволяет системе отвечать на вопросы пользователя даже при отсутствии описывающих фактов в локальной БЗ.

Вариант — 15. Предметная область — музыка (класс сообщений и сущности заданы в лабораторной работе №2, логические правила и фразы — в лабораторной работе №3). Класс сообщения `concept_message_about_entity` относится к сообщениям вида «Что такое ...?», «Кто такой ...?», «Расскажи о ...».

Индивидуальный вариант — вектор экземпляров категорий из четырёх классов (таблица 1.1).

Класс	Выбранное значение	Обоснование
Локальность источника	Внешние сервисы, доступные по сети Internet	Агент обращается к MediaWiki API Википедии по протоколу HTTPS
Структура внешней информации	Полуструктурированные данные: JSON	Ответы MediaWiki API представляются в формате JSON
Модель обработки знаний	Процедурные (алгоритмические) модели	Обработка — пошаговый алгоритм: получить сущность, имя, запрос, разобрать ответ, записать факты
Язык программирования	Python	Агент реализован как Python-модуль `sc-kpm`

Таблица 1.1 — Индивидуальный вариант лабораторной работы

Требуется:

а) спроектировать и реализовать агента, получающего знания из внешнего источника и погружающего их в БЗ;
б) обеспечить вывод новых знаний на основе погруженных: на добавленной заметке должно строиться описание сущности в ответе системы;
в) интегрировать агента с общей программой обработки сообщений NIKA;
г) протестировать работу агента средствами диалога с системой;
д) задокументировать агента: условие инициирования, результат, входы, выходы, процедуры и схему алгоритма.

2 ДОКУМЕНТАЦИЯ АГЕНТА

Агент обогащения музыкальных сущностей (`MusicEntityEnrichmentAgent`) восполняет недостающие факты о сущностях предметной области музыки из Википедии. Если у сущности, названной в вопросе пользователя, в БЗ нет заметки (`nrel_note`), агент находит соответствующую статью Википедии и записывает её вступление в БЗ. Новые знания выводятся на основе уже погруженных сущностей: к узлу сущности добавляются отношения `nrel_note` и (при отсутствии) `nrel_main_idtf` на русском языке, после чего логическое правило ответа строит по ним фразу.

2.1 Условие инициирования и результат

Класс действия — `action_enrich_music_entity` (атомарное действие класса `information_action`), задан в файле `kb/extra/section_subject_domain_of_music/... /concepts/action_enrich_music_entity.scs`.

Инициирование происходит непосредственно из агента стандартного ответа на сообщения (`StandardMessageReplyAgent`, C++): перед генерацией ответа он проверяет, что сообщение принадлежит классу `concept_message_about_entity`, извлекает сущность по роли `rrel_entity` и убеждается, что у сущности нет факта `nrel_note`. Лишь при выполнении всех условий создаётся действие `action_enrich_music_entity` с аргументом — сообщением; C++-агент ожидает завершения обогащения (до 15 с), затем продолжает построение ответа.

Результат работы агента:

- `SC_RESULT_OK` — действие завершено; при успехе поиска у сущности созданы отношения `nrel_note` (вступление статьи, `lang_ru`) и, если отсутствовал, `nrel_main_idtf`; при пропуске (заметка уже есть, сообщение не о сущности, сущность не найдена) — изменения в БЗ не вносятся;
- при сетевых ошибках, таймауте или отсутствии статьи агент логирует предупреждение и также завершает работу успешно — ответ строится по имеющимся в БЗ данным.

2.2 Входы и выходы

Входы агента:

- элемент действия `action_enrich_music_entity`, аргумент по роли `rrel_1` — сообщение класса `concept_message` ∧ `concept_message_about_entity`;
- сущность, связанная с сообщением отношением `rrel_entity`;

- внешний источник — HTTPS-запрос к `ru.wikipedia.org/w/api.php` (формат JSON, таймаут 15 с, заголовок User-Agent «NIKA/1.0»).

Выходы агента:

- SC-структуры в БЗ: `сущность` $\xrightarrow{\text{nrel_note}}$ [`вступление статьи`] (← `lang_ru`); при необходимости — `сущность` $\xrightarrow{\text{nrel_main_idtf}}$ [`заголовок статьи`] (← `lang_ru`);
- статус действия в форме `nrel_status` (успешно / неуспешно), устанавливаемый обёрткой агента.

2.3 Функционал агента

Процедуры агента приведены в таблице 2.1.

Процедура	Назначение
`on_event`	Точка входа по событию действия: запускает `run`, фиксирует статус завершения действия
`run`	Основной алгоритм: проверяет класс сообщения, находит сущность, проверяет наличие `nrel_note`, извлекает имя, выполняет поиск и обогащение
`get_entity_addr`	Поиск сущности: шаблон $message \xrightarrow{rrel_entity} entity$
`get_entity_name`	Получение русского имени сущности по `nrel_main_idtf` с учётом `lang_ru`; иначе — системный идентификатор
`search_wikipedia`	Два запроса к MediaWiki API: `list=search` (заголовок статьи) и `prop=extracts` (вступление); возвращает заголовок и текст либо `None`
`enrich_entity`	Запись в БЗ: при отсутствии русского имени — `nrel_main_idtf` с заголовком статьи; всегда — `nrel_note` с вступлением; оба контейнера помечаются `lang_ru`
`get_ru_idtf`	Проверка наличия русского `nrel_main_idtf` у сущности

Таблица 2.1 — Процедуры агента обогащения сущностей

2.4 Схема алгоритма агента

Алгоритм агента показан на рисунке 2.1.

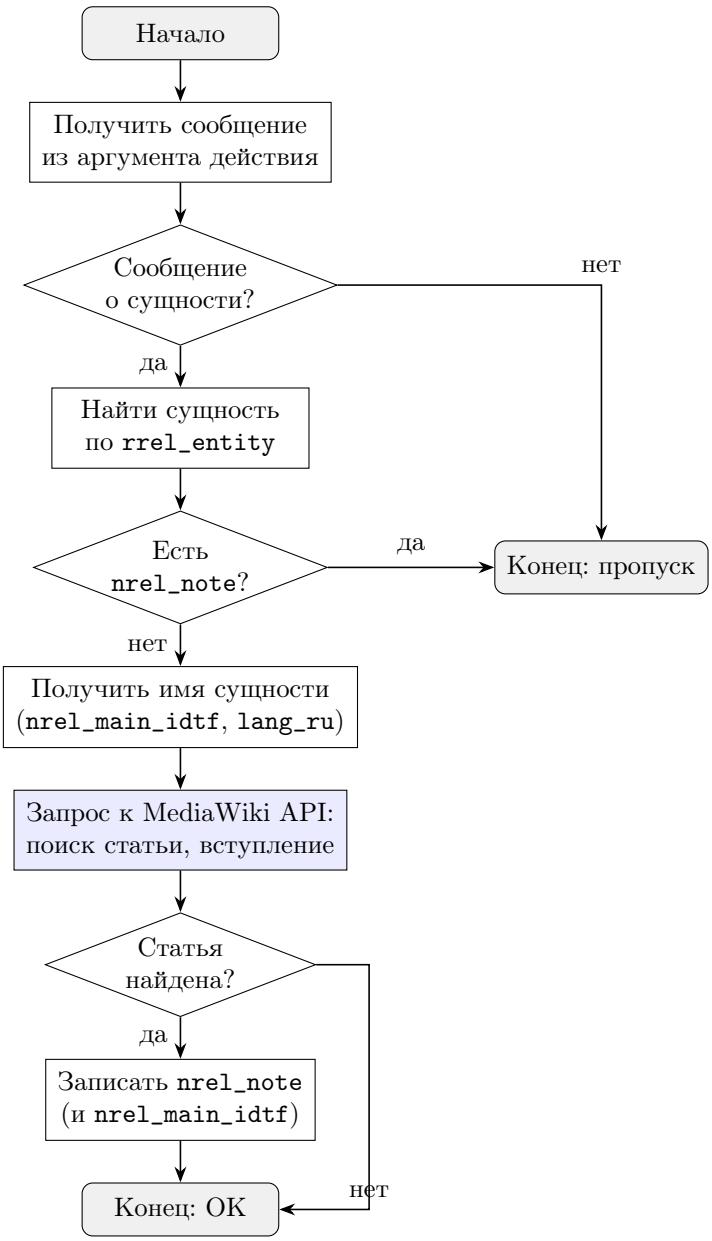


Рисунок 2.1 — Схема алгоритма агента обогащения сущностей

3 ИНТЕГРАЦИЯ И РАБОТА МОДУЛЯ В СИСТЕМЕ

Агент интегрирован в общую программу обработки сообщений NIKA. Python-модуль `MessageProcessingModule` регистрирует `MusicEntityEnrichmentAgent` наравне с агентом прогноза погоды:

```
class MessageProcessingModule(ScModule):
    def __init__(self):
        super().__init__(WeatherAgent(), MusicEntityEnrichmentAgent())
```

Листинг 3.1 — Регистрация агента в Python-модуле

Класс действия `action_enrich_music_entity` описан в БЗ как атомарное действие класса `information_action` с русским и английским идентификаторами (см. раздел 2).

Вызов обогащения встроен в агента стандартного ответа (`StandardMessageReplyAgent.cpp`): если сообщение — о сущности и у сущности нет `nrel_note`, создаётся действие и ожидается его завершение:

```
void StandardMessageReplyAgent::enrichMusicEntityIfNeeded(
    const ScAddr & messageNode)
{
  // is the message about an entity (concept_message_about_entity),
  // does it carry an entity via rrel_entity, and is nrel_note missing?
  if (!isAboutEntityWithMissingNote(messageNode))
    return;
  ScAddr enrichAction = AgentUtils::initAgent(
      &m_memoryCtx, MessageKeynodes::action_enrich_music_entity,
      {messageNode});
  ActionUtils::waitAction(&m_memoryCtx, enrichAction, 15000);
}
```

Листинг 3.2 — Фрагмент StandardMessageReplyAgent: инициирование обогащения

Ключевой фрагмент Python-агента — запрос к MediaWiki API и запись результата в БЗ:

```
search_params = {"action": "query", "list": "search",
                 "srsearch": entity_name, "format": "json"}
resp = requests.get(WIKI_API, params=search_params, timeout=15)
page_title = resp.json()["query"]["search"][0]["title"]
extract_params = {"action": "query", "prop": "extracts",
                  "exintro": True, "explaintext": True,
                  "titles": page_title, "format": "json"}
summary = requests.get(WIKI_API, params=extract_params,
                       timeout=15).json() ... ["extract"]

note_link = create_link(summary, sc_types.LINK_CONST)
note_edge = create_edge(sc_types.EDGE_D_COMMON_CONST,
                        entity_addr, note_link)
create_edge(sc_types.EDGE_ACCESS_CONST_POS_PERM,
            nrel_note, note_edge)
create_edge(sc_types.EDGE_ACCESS_CONST_POS_PERM,
            lang_ru, note_link)
```

Листинг 3.3 — Фрагмент MusicEntityEnrichmentAgent: поиск и запись

Классификация сообщения выполняется модулем witai-shim: намерение `about_entity` определяется по шаблонам «что такое», «кто такой», «расскажи о ...» из файла `classification/intents.json`, сущность распознаётся по словарю `concept_wit_entity` предметной области.

На основе погруженных знаний система выводит ответ: логическое правило `lr_message_about_entity` находит сущность с `nrel_note`, а шаблон фразы `concept_phrase_about_entity` подставляет имя и заметку:

```
1 ${_entity_note_name} - ${_note}.
```

Листинг 3.4 — Шаблон фразы ответа

Полный поток обработки сообщения показан на рисунке 3.1.

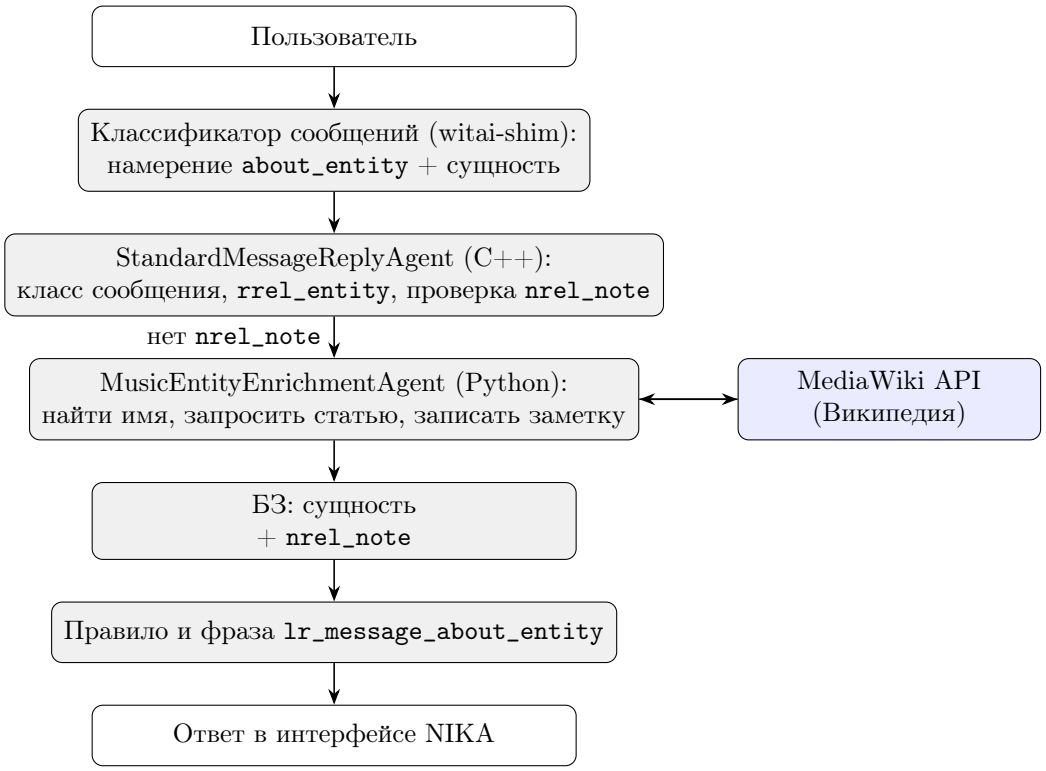


Рисунок 3.1 — Иллюстрация работы модуля адаптации знаний в системе NIKA

4 ТЕСТИРОВАНИЕ

Работа агента проверена диалогом с системой в пользовательском интерфейсе NIKA. Контрольные сущности выбраны так, чтобы были представлены оба случая: заметка отсутствует в локальной БЗ (требуется обогащение из Википедии) и заметка уже есть (обогащение пропускается).

Сценарий тестирования приведён в таблице 4.1. В третьем столбце указано, выполнялось ли обращение к внешнему источнику; в последнем — ожидаемое начало ответа системы (фраза строится шаблоном «имя — заметка»).

Сообщение	Сущность	Обращение к Википедии	Ответ в интерфейсе NIKA (начало)
Что такое Thriller?	`thriller_album`	да: статья «Thriller»	Thriller — шестой студийный альбом американского музыканта Майкла Джексона …
Что такое Bohemian Rhapsody?	`bohemian_rhapsody`	нет: `nrel_note` есть в БЗ	Bohemian Rhapsody — песня группы Queen (1975) из альбома A Night at the Opera …
Что такое Волшебная флейта?	`the_magic_flute`	да: статья «Волшебная флейта»	Волшебная флейта — опера-зингшпиль Моцарта в 2-х действиях …
Что такое The Dark Side of the Moon?	`the_dark_side_of_the_moon`	да: статья «The Dark Side of the Moon»	The Dark Side of the Moon — восьмой студийный альбом британской рок-группы Pink Floyd …
Что исполнил Thriller?	`thriller_album`	нет: заметка уже погружена	Thriller — шестой студийный альбом … (повторное обращение пропущено)

Таблица 4.1 — Тестирование агента обогащения сущностей

Диалог пользователя с системой зафиксирован в пользовательском интерфейсе NIKA (рисунок 4.1).



Рисунок 4.1 — Диалог обогащения сущностей в интерфейсе NIKA

Работа агента подтверждена журналами системы: в журнале problem-solver видны шаги «enriching music entity from Wikipedia» и «Trying to generate by formula: lr_message_about_entity», в журнале py-sc-server — «Searching Wikipedia for: Thriller», «Found Wikipedia page: Thriller», «Entity enriched successfully». Повторный вопрос к уже обогащенной сущности вызывает пропуск обогащения: агент фиксирует наличие `nrel_note` и сразу передаёт управление построению ответа.

ЗАКЛЮЧЕНИЕ

В лабораторной работе разработан модуль адаптации знаний для системы NIKA — агент `MusicEntityEnrichmentAgent` на языке Python. Агент получает знания из внешнего источника (MediaWiki API Википедии, формат JSON), обрабатывает их процедурной моделью и погружает в базу знаний: к сущности без описания добавляются отношения `nrel_note` и при необходимости `nrel_main_idtf`. На основе погруженных знаний система выводит новые — логическое правило и шаблон фразы строят ответ на вопрос пользователя «Что такое ...?» из заметки сущности.

Агент интегрирован с общей программой обработки сообщений: классификация сообщения о сущности выполняется модулем witai-shim, инициирование обогащения — агентом стандартного ответа `StandardMessageReplyAgent`, регистрация — модулем `MessageProcessingModule`. Обогащение выполняется один раз: при наличии `nrel_note` в БЗ обращения к внешнему источнику не происходит.

Тестирование в диалоге показало, что сущности, отсутствующие в локальной БЗ с описанием (альбом Thriller, «Волшебная флейта», «The Dark Side of the Moon»), получают заметку из Википедии и служат основой ответа системы, а сущности с готовой заметкой (Bohemian Rhapsody) обогащению не подвергаются.

Индивидуальный вариант: внешние сервисы (Internet) / JSON / процедурные модели / Python.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] NIKA: Intelligent Knowledge-driven Assistant [Электронный ресурс]. — Режим доступа: `https://github.com/ostis-apps/nika`. — Дата доступа: 15.10.2026.

[2] OSTIS: Open Semantic Technology for Intelligent Systems [Электронный ресурс]. — Режим доступа: `https://ostis.net/`. — Дата доступа: 15.10.2026.

[3] SC-код. Руководство пользователя [Электронный ресурс]. — Режим доступа: `https://ostis-dev.github.io/sc-machine/`. — Дата доступа: 15.10.2026.

[4] MediaWiki API [Электронный ресурс]. — Режим доступа: `https://www.mediawiki.org/wiki/API:Main\_page`. — Дата доступа: 15.10.2026.